Milestone 4

Model Training

Contents:

Overview / Objective

- This fourth milestone focuses on the **implementation of the complete AI Stylist architecture**, as outlined in the previously demonstrated flowchart. It builds upon the work from earlier milestones to integrate all components — from data preprocessing to query handling and product recommendation — into a functional end-to-end system.
- This document provides:
    - Detailed information on the **dataset** used for catalog, customer, and transaction data.
    - A description of the **system architecture** and its workflow.

    - The **initial results** obtained after implementing the architecture.
    - **Observations** and notes for further improvement in the next milestone.
    - An **update on changes** made since the previous milestone.
    - The **division of work** among team members for this stage.
    - And finally, a **summary of the previous milestones**, including data preprocessing and architecture design progress.
- **Link to Previous Milestones** [Link]

Dataset Details

- Description of datasets:
    - **Catalog** (before preprocessing):
        - File name: “articles.csv”
        - No. of features: 25
        - No. of samples: 105542
        - Feature description:
            - article_id
            - product_code
            - prod_name
            - product_type_no
            - product_type_name
            - product_group_name
            - graphical_appearance_no
            - graphical_appearance_name
            - colour_group_code
            - colour_group_name
            - perceived_colour_value_id
            - perceived_colour_value_name
            - perceived_colour_master_id
            - perceived_colour_master_name
            - department_no

- department_name
- index_code
- index_name
- index_group_no
- index_group_name
- section_no
- section_name
- garment_group_no
- garment_group_name
- detail_desc

- **Catalog** (preprocessed):
  - File name: “final_cloth_data_csv.csv”
  - No. of features: 18
  - No. of samples: 78,017
  - Feature description:
    - **SKU_No:** Unique identifier assigned to each product.
    - **prod_name:** The product’s display or short name.
    - **product_type_name:** The specific type or model classification of the product.
    - **product_group_name:** The broader category under which the product falls.
    - **graphical_appearance_name:** The visual design or pattern category of the product.
    - **colour_group_name:** The primary color classification or group name.
    - **department_name:** The department or collection to which the product belongs.
    - **index_name:** Internal index categorizing the product within the department.
    - **index_group_name:** Higher-level grouping for similar index categories.

- **section_name:** Store or catalog section (e.g., by gender, style, or use).
- **garment_group_name:** Group defining the garment's construction or functional type.
- **detail_desc:** Detailed description of the product's material, design, or features.
- **Eco_Score:** A numerical score representing the product's environmental sustainability.
- **Style_Reward_Points:** Loyalty or style reward points earned upon purchase.
- **Stock_Status:** Current availability of the product (e.g., "In Stock", "Low Stock").
- **Price_INR:** Product's price in Indian Rupees (₹).
- **Discount_Percentage:** Discount offered on the product (in percentage).
- **Return_Type:** Return policy applicable to this product.
- **Image_URL:** Path or URL of the product image for display.

- Download link

- **Transactions data:**
  - File name: "transactions_filtered_390k.csv"
  - No. of features: 9
  - No. of samples: 3,90,000
  - Feature description:
    - **t_dat:** Transaction date representing when the purchase was made.
    - **article_id:** Unique identifier for the specific article or item sold.
    - **sales_channel_id:** Channel through which the sale occurred (e.g., 1 = Online, 2 = In-store).
    - **payment_method:** The payment method used for the transaction (e.g., Credit Card, UPI, Cash).

- **SKU_No:** Unique identifier for the product (links to the product catalog).
- **prod_return_type:** Type of return policy associated with the product.
- **return_Status:** Indicates whether the item was returned or not.
- **delivered:** Delivery status of the product.
- **cust_id:** Unique identifier for the customer who made the purchase.

- Download link

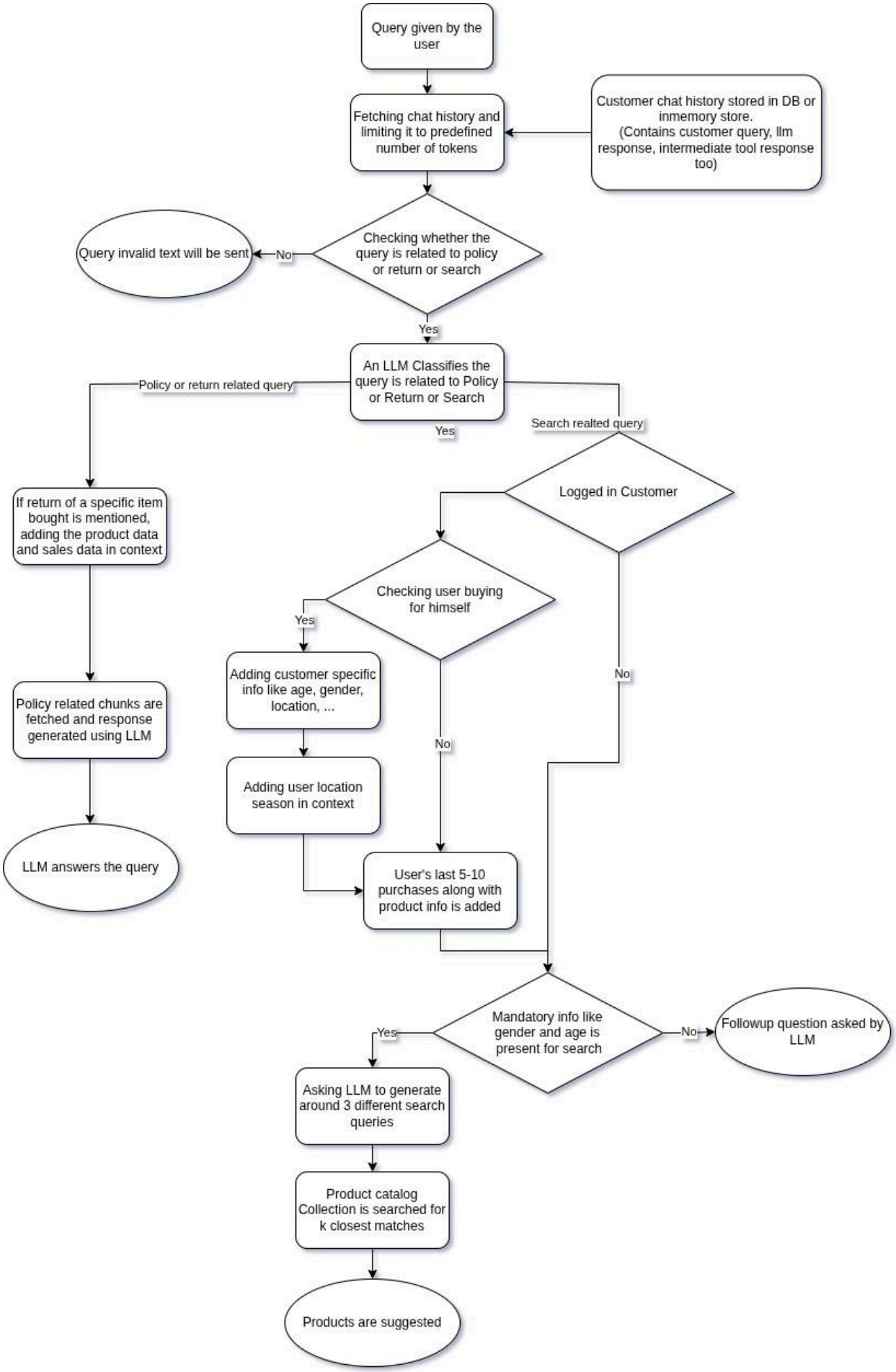
- **Customers data:**
  - File name: "customers_matched_390k.csv"
  - No. of features: 6
  - No. of samples: 1,92,508
  - Feature description:
    - **age:** Age range of the customer.
    - **postal_code:** The city or postal area associated with the customer's address.
    - **gender:** Gender identity of the customer.
    - **size:** Preferred clothing size of the customer.
    - **cust_id:** Unique identifier assigned to each customer.
    - **embed_text:** Textual embedding or descriptive summary combining key customer attributes for use in text-based or embedding models.
  - Download link

- **Image Data:**
  - Resolution: 1166 × 1750 Pixels
  - No. of images: 77000

    - Download link

Architecture

- The flow chart could be seen below the architecture details.
- It goes as follows:
    1. The user inputs a text query.
    2. The system retrieves recent chat history from the memory.py file and context from the database or memory store.
    3. The query is analyzed to determine whether it relates to **policy**, **return**, or **product search**.
    4. If it's a **policy or return** query, the system adds relevant product and transaction data before generating an LLM-based response.
    5. If it's a **search** query, user-specific details such as age, gender, and location are added to personalize results.
    6. Based on this enriched context, the LLM generates optimized search queries.
    7. The product catalog is then searched for the top **k closest matches**, and relevant products are suggested to the user.
- Flow chart:


Query given by the user
Fetching chat history and limiting it to predefined number of tokens
Customer chat history stored in DB or inmemory store. (Contains customer query, llm response, intermediate tool response too)
Checking whether the query is related to policy or return or search
No
Query invalid text will be sent
Yes
An LLM Classifies the query is related to Policy or Return or Search
Policy or return related query
Yes
Search realted query
If return of a specific item bought is mentioned, adding the product data and sales data in context
Logged in Customer
Checking user buying for himself
Yes
No
Adding customer specific info like age, gender, location, ...
Policy related chunks are fetched and response generated using LLM
Adding user location season in context
No
LLM answers the query
User's last 5-10 purchases along with product info is added
Mandatory info like gender and age is present for search
Yes
No
Followup question asked by LLM
Asking LLM to generate around 3 different search queries
Product catalog Collection is searched for k closest matches
Products are suggested

Initial Results

- When using minimal data (tested with minimal data first, because it was not always feasible to test on the whole data) while testing the results were not satisfactory.
- Without building context and without any followup questions the search results were generic and bland, and intended products were not fetched.
- When the user query spammed multiple queries the search results were losing its relevance. The model seemed to be struggling.

Update on Changes Made

- We focused on context building where without age and gender info, the product search will not be done.
- Also, we found that chat history needs to be maintained and fed into the llm for better understanding of customer intention and better search quality.

Work Division

- **Image data management - Neeraj and Harsha**
    - The data consists of both textual and image data, totalling to 30 GB. And, since it is not possible to put it on a G-Drive, Neeraj has downloaded the whole dataset from the website and kept it on his machine.
    - He has segregated 5 GBs of it for us to test it out while working on our individual parts, and after integrating it together, he is able to perform the whole pipeline on his machine locally.
- **Policy RAG and streamlit UI - Madhav and Manish**
    - Embedding policy doc,
    - Fetching chunks from policy doc
    - Prompt engineering for getting answer

- **Customer context building - Harsha and Manish**
    - Fetches customer meta from csv and build context like age, gender, location
    - If not a logged in customer then followup questions are asked
- **Product RAG - Harsha and Manish**
    - Classify query whether this is policy related or product search related - Manish
    - Product fetching from embedding
    - Product reranker chain
- **Product Embedding - Neeraj and Adarsh**
    - Embedding product meta and saving it in VDB
- **Memory and Customer session Management - Manish**
    - Memory and customer meta is maintained in a class instance
- **Preparing the submission document - Adarsh, Manish and Harsha**

Summary of previous Milestones

Milestone 1:

- This milestone consists of the problem definition and literature review. The project objective was divided into two parts, A and B. The first one being “Store policy + Customer help desk” and the other part is an “AI Stylist”.
- It then talks about the existing solutions, baselines and benchmarks. Taking a deep dive into them.
- Concluding that our suggestion system is going to use text features alone and will suggest clothes based on user inputs.
- The folder structure in as given below:
    - /group-5-DS-AI-Lab-Project
        - /milestones
            - /milestone-1
                - Milestone - 1.pdf

Milestone 2:

- This milestone had details about us choosing a dataset, and performing analysis on it.

- It had some sample images from the image dataset.
- The code for the EDA performed is also provided in a jupyter notebook.
- The FAQ and Policy details are provided in a pdf file with the same name.
- And the dataset for articles, customers detail and customers' purchase histories have been shared.
- The folder structure is as given below:
    - /group-5-DS-AI-Lab-Project
        - /milestones
            - /milestone-2
                - /sample_images
                    - (a few images are here)
                - EDA_on_catalog.ipynb
                - Milestone - 2.pdf
                - articles.csv
                - customers.csv
                - faq_and_policy.pdf
                - purchase_history.xlsx

Milestone 3:

- The third milestone had intricate details about the RAG (Retrieval Augmented Generation) method and a handful of embedding models to compare their strengths and weaknesses.
- A jupyter notebook showing the hands-on code for the whole embedding pipeline.
- It starts the work with "filtered_data_with_images.csv" containing the textual details about the images, and embeds it using the "all-MiniLM-L6-v2" model.
- These vectors and the meta data are then saved for later use.
- And, then using the FAISS (Facebook AI Similarity Search) to deal with the search queries.
- Finally concluding with some sample queries and their results.
- The folder structure is as given below:
    - /group-5-DS-AI-Lab-Project
        - /milestones

- /milestone-3
    - EmbeddingTest.ipynb
    - Milestone - 3.pdf